

Heart Disease Dataset — Step-by-Step Solution Guide

Heart Disease Dataset: Step-by-Step Solution Guide

This document walks through how the heart disease prediction model was built, step by step, and explains the reasoning behind each decision.

Step 1: Load and Look at the Data

```
df = pd.read_csv("heart_disease_uci.csv")
```

Before touching any model, inspect what you have: `df.info()`, `df.isnull().sum()`, `df['num'].value_counts()`. This shows the shape of the data, what's missing, and how balanced the target is. Skipping this step is one of the most common reasons people build bad models — it leads to wrong assumptions about the data.

Step 2: Define the Target Clearly

The raw `num` column had 5 classes (0 = no disease, 1-4 = increasing severity). This was collapsed into a binary target:

```
df['num'] = (df['num'] > 0).astype(int)
```

Why: with only about 920 rows, splitting into 5 classes leaves too few examples per class to learn from reliably. Binary (disease / no disease) is also the more useful real-world question to answer.

Step 3: Remove Columns That Shouldn't Be There

```
df.drop(columns=['id', 'dataset'], inplace=True)
```

- **id** is just a row number — zero predictive value, pure noise.
- **dataset** (hospital name) is a form of **data leakage**. It correlates with the label for reasons that have nothing to do with a patient's actual health — different hospitals had different reporting habits and missing data patterns. A model that learns “Cleveland patients tend to be lower risk” isn't learning cardiology, it's memorizing a dataset quirk.

General lesson: for every column, ask “would this be available and meaningful for a *new* patient, and does it reflect the thing I'm actually trying to predict?” If the answer is no, drop it.

Step 4: Split Features into Numeric vs. Categorical

```
numerical_col = ['age', 'trestbps', 'chol', 'thalch', 'oldpeak', 'ca']  
nominal_cat   = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope',  
                 'thal']
```

These need different treatment — numbers can be scaled and averaged, text categories cannot.

Step 5: Handle Missing Values (Imputation)

- Numeric columns: fill gaps with the **median** (robust to outliers, unlike the mean).
- Categorical columns: fill gaps with the **most frequent value**.

```
SimpleImputer(strategy='median')           # numeric columns  
SimpleImputer(strategy='most_frequent')   # categorical columns
```

Why not just drop rows with missing values? Columns like `ca` and `thal` were missing in 30-50% of rows. Dropping those rows would throw away most of the dataset.

Step 6: Encode Categorical Text into Numbers

Models only understand numbers. `OneHotEncoder` turns a column like `sex: Male/Female` into two columns, `sex_Male: 0/1` and `sex_Female: 0/1`, so the model can use the information without assuming a false numeric order.

Step 7: Scale Numeric Features

```
StandardScaler() # centers each feature to mean 0, standard deviation  
1
```

Tree-based models like Random Forest don't strictly need this, but models like SVM and Logistic Regression do. Scaling everything up front makes the same pipeline reusable across model types without rewriting it.

Step 8: Chain Everything into a Pipeline

```
preprocessor = ColumnTransformer([  
    ('numerical', num_pipeline, numerical_col),  
    ('nominal', cat_pipeline, nominal_cat),  
])
```

Why use a Pipeline instead of preprocessing the whole dataframe by hand? It prevents **leakage from the test set into the training process** — the imputer and scaler learn their median/most-frequent-value/mean only from the training fold each time, never peeking at the test data. Manually preprocessing the full dataset before splitting it is a classic mistake that quietly inflates test accuracy.

Step 9: Split into Train and Test Sets

```
train_test_split(x, y, test_size=0.2, stratify=y, random_state=42)
```

`stratify=y` keeps the same disease / no-disease ratio in both the training and test sets. `random_state` makes the split reproducible.

Step 10: Try Multiple Models, Not Just One

Several model families were compared: Random Forest, Extra Trees, Gradient Boosting, SVM, and Logistic Regression. Different models pick up on different kinds of patterns, and it isn't obvious in advance which will perform best. Random Forest came out on top for this dataset.

Step 11: Tune Hyperparameters with Cross-Validation

```
RandomizedSearchCV(pipe, param_distributions, n_iter=40, cv=5,  
scoring='accuracy')
```

Instead of relying on a single train/test split (which can be noisy — lucky or unlucky depending on which rows land where), `cv=5` trains and validates the model five times on different slices of the training data and averages the results. This gives a much more reliable accuracy estimate.

`RandomizedSearchCV` samples random combinations from the hyperparameter grid rather than testing every single one, which is faster than `GridSearchCV` while finding a similarly good setting.

Step 12: Evaluate Properly on the Held-Out Test Set

Accuracy alone isn't enough. A full evaluation includes:

- **Confusion matrix** — shows exactly which patients were misclassified and in which direction. A false negative (predicting “no disease” when disease is present) is the more dangerous kind of error in this domain.

- **Precision / Recall / F1** — especially important if the classes are not perfectly balanced.
- **ROC AUC** — measures how well the model ranks patients by risk across all possible thresholds, not just one fixed cutoff.
- **Train vs. test accuracy gap** — a large gap (for example, 97% train vs. 86% test) signals the model memorized the training data rather than learning generalizable patterns.

Step 13: Save the Model

```
joblib.dump(best_model, "model.joblib")
```

This allows the trained model to be reused on new patients without retraining from scratch.

The Habit That Mattered Most

Before any modeling begins, always ask: **is this column real signal, or is it leaking the answer?** Tuning hyperparameters typically buys a few percentage points of accuracy. Catching a leak — like the `dataset` column in this case — is what makes the difference between a model that merely looks good on paper and one that actually works on new, unseen patients.

Summary of Results

Metric	Original Model	Improved Model
Test Accuracy	85.9%	87.5%
Train Accuracy	96.9%	91.3%
Train/Test Gap	11.0 pts	3.8 pts
<code>dataset</code> column used	Yes (leakage)	No
ROC AUC	Not measured	0.927

The improved model is both more accurate and more trustworthy: it removed a leaking feature, generalizes better (smaller train/test gap), and still scored higher on unseen data.